SERVER-SIDE MODULE

CONTENTS

This module has the following files:

- MODULE_SERVER_SIDE.doc
- MODULE_SERVER_SIDE_MEDIA.zip

INTRODUCTION

Starlight Observatory is a world-class astronomy center hosting **rare celestial observation events**. Due to **limited telescope capacity and high public demand**, the observatory requires a reliable online reservation platform. You are appointed as a competent developer to build the Starlight Observatory System, enabling society members to register, validate their membership, explore astronomical slots, and reserve telescope sessions securely.

DESCRIPTION OF PROJECTS AND TASKS

Registration involves the society platform (registering/logging in to the platform and applying for observatory slot reservations), the observatory company platform (viewing and accepting/rejecting validations), and the officer platform (updating telescope data, checking validity of society data).

This module is divided into two phases. In the first phase, you will create a REST API. In the second phase, you will build a frontend application. You must use the provided frameworks:

- Laravel (v11.x)
- React (v18.x with react-router-dom and axios) or Vue (v3.x with vue-router and axios)

Phase one – Society Observatory REST API

Authentication

Before a society member performs any activity on the platform, they must log in using their username and password. **Sanctum** should be used to create tokens. The generated token will be used to access all available protected endpoints.

Specifications:

Feature: A1 - Login and Logout as society: as a visitor, i want to login so that i can enter the system and apply for Observatory Reservations.

Scenario: Society Login

Request:

- URL: `[domain]/api/v1/auth/login`
- Method: `POST`
- Body: {

```
"username": "johndoe",
"password": "password123"
}
```

Response:

- **A1a - If login success**
  - Header: Response code: `200`
  - Body: {

```
"name": "John Doe",
"username": "johndoe",
"member_type": "researcher",
"born_date": "2002-05-14",
"gender": "male",
"address": "Jl. Raya Singosari No. 12, Malang",
"token": "1|qgV2cTisoRVSwX6R2iPpxd...",
"regional": {
  "id": 1,
  "province": "Jawa Timur",
  "district": "Kabupaten Malang"
```

```
  }
}
```

- **A1b - If Username or Password incorrect**
  - Header: Response code: `401`
  - Body: {
    ```
    "message": "Username or Password incorrect"
    }
    ```

Scenario: Society Logout

Request:

- URL: `[domain]/api/v1/auth/logout`
- Method: `POST`
- Header: `{ Authorization: Bearer xxxxxx }`

Response:

- **A1c - If logout success**
  - Header: Response code: `200`
  - Body: {
    ```
    "message": "Logout success"
    }
    ```

- **A1d - If logout invalid token**
  - Header: Response code: `401`
  - Body: {
    ```
    "message": "Unauthorized user"
    }
    ```

Request Data Validation

To be able to apply for observatory reservations, the society member must perform data validation on the platform first. Society can view validity status and make validation requests. Society can only make one validation request.

Specifications:

Feature: A2 - Request Data Validation: As a society, I want to request data validation so that I can apply for observatory reservations when my data validation is accepted.

Scenario: Request data validations

Request:

- URL: `[domain]/api/v1/validation`
- Method: `POST`
- Header: `{ Authorization: Bearer xxxxxx }`
- Body:

```
{
  "institution": "Universitas Negeri Malang",
  "occupation": "Astronomy Researcher",
  "reason_accepted": "Need high precision optical telescope for
Comet Halley research"
}
```

Response:

- **A2a - If request data validation success**
  - Header: Response code: `200`
  - Body: `{ "message": "Request data validation sent successful" }`

- **A2b - If invalid token**
  - Header: Response code: `401`
  - Body: `{ "message": "Unauthorized user" }`

Scenario: Get society data validation

Request:

- URL: `[domain]/api/v1/validations`
- Method: `GET`
- Header: `{ Authorization: Bearer xxxxxx }`

Response:

- **A2c - If success**

    - Header: Response code: `200`
    - Body: {

```
  "validation": {
    "id": 1,
    "status": "pending",
    "institution": "Universitas Negeri Malang",
    "occupation": "Astronomy Researcher",
    "reason_accepted": "Need high precision optical
telescope for Comet Halley research",
    "validator_notes": null,
    "validator": null
  }
}
```

(Status enum values: `pending` | `accepted` | `rejected`)

- **A2d - If invalid token**
    - Header: Response code: `401`
    - Body: {

```
  "message": "Unauthorized user"
}
```

Observatory Slots

When the validation data has been accepted by the validator, society can view and search available observatory slots based on dynamic query parameters.

Specifications:

Feature: A3 – Observatory Slots: As a society, I want to see a list of observatory slots.

Scenario: Get all Observatory Slots

Request:

- URL: `[domain]/api/v1/slots`
- Method: `GET`
- Header: `{ Authorization: Bearer xxxxxx }`
- Params: `search`, `type`, `min_price`, `max_price` (Optional query chaining)

Response:

- **A3a - If success**
  - Header: Response code: `200`
  - Body: {

```
"slots": [
  {
    "id": 1,
    "event_name": "Halley Comet Passage 2026",
    "event_date": "2026-11-15",
    "price_per_slot": "150000",
    "max_capacity": 50,
    "booked_count": 12,
    "telescope": {
      "id": 1,
      "name": "Zeiss Major Optical Telescope",
      "type": "optical"
    }
  }
]
}
```

- **A3b - If invalid token**
  - Header: Response code: `401`
  - Body: {

```
"message": "Unauthorized user"
}
```

Scenario: Get Slot detail by ID

Request:

- URL: `[domain]/api/v1/slots/<id>`
- Method: `GET`
- Header: `{ Authorization: Bearer xxxxxx }`

Response:

- **A3c - If success**
  - Header: Response code: `200`
  - Body: {

```
"slot": {
```

```
        "id": 1,
        "event_name": "Halley Comet Passage 2026",
        "event_date": "2026-11-15",
        "price_per_slot": "150000",
        "max_capacity": 50,
        "booked_count": 12,
        "description": "High magnification observation of
    Comet Halley nucleus.",
        "telescope": {
          "id": 1,
          "name": "Zeiss Major Optical Telescope",
          "type": "optical"
        }
      }
    }
```


- **A3d - If invalid token**
    - Header: Response code: `401`
    - Body: {

      ```
        "message": "Unauthorized user"
      }
      ```


Applying for Reservation

The society can apply for an observatory slot reservation if their validation data has been accepted by the validator. Application for a reservation can only be once per active slot.

Specifications:

Feature: A4 – Applying for Reservation: as a society, i want to apply for observatory slot reservation.

Scenario: Applying for reservation

Request:

- URL: `[domain]/api/v1/reservations`
- Method: `POST`
- Header: `{ Authorization: Bearer xxxxxx }`
- Body:

```
{
  "observatory_slot_id": 1,
  "requested_tickets": 2,
  "notes": "Observation for astrophysics academic paper"
}
```

Response:

- **A4a - If success**
  - Header: Response code: `200`
  - Body: {
    ```
    "message": "Applying for reservation successful"
    }
    ```
- **A4b - If invalid token**
  - Header: Response code: `401`
  - Body: {
    ```
    "message": "Unauthorized user"
    }
    ```
- **A4c - If society validation data hasn't been accepted by validator**
  - Header: Response code: `401`
  - Body: {
    ```
    "message": "Your data validation must be accepted by
    validator before"
    }
    ```
- **A4d - If invalid fields**
  - Header: Response code: `401`
  - Body: {
    ```
    "message": "Invalid field",
    "errors": {
      "observatory_slot_id": [
        "The observatory slot id field is required."
      ],
      "requested_tickets": [
        "The requested tickets field is required."
      ]
    ```

```
  }
}
```

- **A4e - If have applied before**
  - Header: Response code: `401`
  - Body: {
    `"message": "Application for a reservation can only be once"`
    }

- **A4f - If ticket quota exceeded (Pessimistic Locking / Atomic Failure)**
  - Header: Response code: `422`
  - Body: {
    `"message": "Requested ticket quantity exceeds available slot capacity"`
    }

Scenario: Get all of society reservation applications

Request:

- URL: `[domain]/api/v1/reservations`
- Method: `GET`
- Header: `{ Authorization: Bearer xxxxxx }`

Response:

- **A4g - If success**
  - Header: Response code: `200`
  - Body: {

```
"reservations": [
  {
    "id": 1,
    "booking_code": "OBS-20261101-001",
    "total_tickets": 2,
    "total_price": "300000",
    "status": "pending",
    "notes": "Observation for astrophysics academic paper",
    "slot": {
```

```
            "id": 1,
            "event_name": "Halley Comet Passage 2026",
            "event_date": "2026-11-15",
            "telescope": "Zeiss Major Optical Telescope"
          }
        }
      ]
    }
```

- **A4h - If invalid token**
  - Header: Response code: `401`
  - Body: `{`
    ```
      "message": "Unauthorized user"
    }
    ```

INSTRUCTION FOR COMPETITORS

- You may create additional endpoints if necessary, but you must fulfil all of the requirements listed above.
- Create a root folder called `XX_SERVER_MODULE` in your local computer, where `XX` is your computer number.
- Created Postman Collection, Database (`db-dump.sql`), and ER diagram (`db-diagram.pdf`) should be exported and saved under the root folder `<host>/XX_SERVER_MODULE`.
- REST API should be reached on `<host>/XX_SERVER_MODULE/BACKEND`.
- Frontend app should be reached on `<host>/XX_SERVER_MODULE/FRONTEND`.